

Покрокова інструкція виконання фінального проєкту

Технічні вимоги

Інфраструктура: AWS з використанням Terraform

Компоненти: VPC, EKS, RDS, ECR, Jenkins, Argo CD, Prometheus, Grafana

Етапи виконання

1. Підготовка середовища:

Ініціалізувати Terraform.

Перевірити всі необхідні змінні та параметри.

2. Розгортання інфраструктури:

Виконати команду розгортання:

```
terraform apply
```

Перевірити стан ресурсів через:

```
```\nkubectl get all -n jenkins\nkubectl get all -n argocd\nkubectl get all -n monitoring\n```\n
```

## 3. Перевірка доступності:

Jenkins:

```
kubectl port-forward svc/jenkins 8080:8080 -n jenkins
```

Argo CD:

```
kubectl port-forward svc/argocd-server 8081:443 -n argocd
```

## 4. Моніторинг та перевірка метрик:

Grafana:

```
kubectl port-forward svc/grafana 3000:80 -n monitoring
```

Перевірити стан метрик в Grafana Dashboard.

### ⚠️ УВАГА!

⚠️ При роботі з хмарними провайдерами завжди пам'ятайте: **невикористані ресурси можуть призвести до значних витрат**. Щоб уникнути непередбачуваних рахунків, після перевірки вашого коду **обов'язково видаляйте створені ресурси**. Використовуйте команду `terraform destroy`.

### УВАГА! ⚠️

**Пам'ятайте порядок запуску інфраструктури після видалення!** При видаленні всієї інфраструктури за допомогою `terraform destroy` ви також видаляєте S3-бакет і DynamoDB-таблицю, які використовуються для збереження **Terraform стейту**.

## Критерії оцінювання та прийняття

Критерії прийняття фінального проєкту є обов'язковою умовою його оцінювання ментором. Якщо якийсь із критеріїв не виконано, ментор відправить завдання на доопрацювання без оцінювання. Якщо вам «тільки уточнити» 😊 або ви застопорилися на якомусь з етапів виконання — звертайтеся до ментора у Slack).

Оцінювання від 0 до 100 балів.

Створено середовище з коректною архітектурою (20 балів)

Налаштовано безпеку через VPC, IAM, Security Groups (20 балів)

Розгорнуто застосунок в AWS з CI/CD (30 балів)

Налаштовано моніторинг та автомасштабування (20 балів)

Коректне оформлення та зрозумілість документації (10 балів)

Якщо є помилки або недоліки виконання, кількість балів на кожному етапі зменшується пропорційно до наявних помилок на розсуд ментора.

## Формат здачі

1. Посилання на ваш GitHub-репозиторій із гілкою (гілка `final project`).
2. Прикріплені файли репозиторію у форматі `zip` із назвою `final_DevOps_ПІБ`.

Проект має мати таку структуру:

```
Project/
├── main.tf # Головний файл для підключення модулів
├── backend.tf # Налаштування бекенду для стейтів (S3 +
DynamoDB
├── outputs.tf # Загальні виводи ресурсів
├── modules/ # Каталог з усіма модулями
│ ├── s3-backend/ # Модуль для S3 та DynamoDB
│ │ ├── s3.tf # Створення S3-бакета
│ │ ├── dynamodb.tf # Створення DynamoDB
│ │ ├── variables.tf # Змінні для S3
│ │ └── outputs.tf # Виведення інформації про S3 та DynamoDB
│ ├── vpc/ # Модуль для VPC
│ │ ├── vpc.tf # Створення VPC, підмереж, Internet Gateway
│ │ ├── routes.tf # Налаштування маршрутизації
│ │ ├── variables.tf # Змінні для VPC
│ │ └── outputs.tf
│ ├── ecr/ # Модуль для ECR
│ │ ├── ecr.tf # Створення ECR репозиторію
│ │ ├── variables.tf # Змінні для ECR
│ │ └── outputs.tf # Виведення URL репозиторію
│ ├── eks/ # Модуль для Kubernetes кластера
│ │ ├── eks.tf # Створення кластера
│ │ ├── aws_ebs_csi_driver.tf # Встановлення плагіну csi drive
│ │ ├── variables.tf # Змінні для EKS
│ │ └── outputs.tf # Виведення інформації про кластер
│ ├── rds/ # Модуль для RDS
│ │ ├── rds.tf # Створення RDS бази даних
│ │ ├── aurora.tf # Створення aurora кластера бази даних
│ │ ├── shared.tf # Спільні ресурси
│ │ ├── variables.tf # Змінні (ресурси, креденшели, values)
│ │ └── outputs.tf
│ ├── jenkins/ # Модуль для Helm-установки Jenkins
│ │ ├── jenkins.tf # Helm release для Jenkins
│ │ ├── variables.tf # Змінні (ресурси, креденшели, values)
│ │ ├── providers.tf # Оголошення провайдерів
│ │ ├── values.yaml # Конфігурація jenkins
│ │ └── outputs.tf # Виводи (URL, пароль адміністратора)
│ └── argo_cd/ # ✅ Новий модуль для Helm-установки Argo CD
│ ├── jenkins.tf # Helm release для Jenkins
│ ├── variables.tf # Змінні (версія чарта, namespace, repo URL
тощо)
│ └── providers.tf # Kubernetes+Helm. переносимо з модуля
jenkins
```

```

├── values.yaml # Кастомна конфігурація Argo CD
├── outputs.tf # Виводи (hostname, initial admin password)
│ └── charts/ # Helm-чарт для створення app'ів
│ ├── Chart.yaml
│ └── values.yaml # Список applications,
repositories
│ └── templates/
│ ├── application.yaml
│ └── repository.yaml
├── charts/
│ ├── django-app/
│ │ ├── templates/
│ │ │ ├── deployment.yaml
│ │ │ ├── service.yaml
│ │ │ ├── configmap.yaml
│ │ │ └── hpa.yaml
│ │ ├── Chart.yaml
│ │ └── values.yaml # ConfigMap зі змінними середовища
└── Django
 ├── app\
 ├── Dockerfile
 ├── Jenkinsfile
 └── docker-compose.yaml

```